Auditable Agents

Master of Engineering (Computer Science and Engineering) — software workspace for a tamper-evident audit trail of LLM agent actions, anchored on a blockchain.

Author: Manikandan Sankaranarayanan · S Veersamy Chettiya College of Engineering · smak07.s@gmail.com
Working title: Auditable AI Agents: An Immutable On-Chain Audit Trail for LLM Agent Actions

Documentation index: `docs/README.md` (start with `docs/thesis/technologies.md` and `docs/TESTING.md`; every document also has a PDF).
Concepts and plan: `docs/proposal/`. Locked choices: `docs/proposal/decisions.md`.
Author profile: `references/personal-info.md`.

Idea in one paragraph

An LLM agent's prompts, tool calls and results are logged as hash-chained entries. Batches of entry hashes are folded into a Merkle root and anchored on a blockchain. A verifier recomputes the hashes from the stored logs and compares them with the chain, so any edit, deletion, insertion or reordering is detected and located. Only hashes go on-chain; full logs stay off-chain.

Quick start (Month 1)

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python -m pytest -q          # 79 Python tests

# Smart contract tests on a local Hardhat chain
npm install
python scripts/make_vectors.py   # regenerate Python-computed test vectors
npx hardhat test              # 18 contract tests

# End-to-end demo: agent -> audit log -> anchor on a local chain -> tamper -> detect
npx hardhat compile
python scripts/run_demo.py                      # scripted stand-in LLM, runs offline

# Same demo with a real local LLM (no API key): install Ollama, then
brew install ollama && ollama serve &   # or: brew services start ollama
ollama pull qwen3:8b
python scripts/run_demo.py --llm ollama

# Or with Claude; the key is read from the environment
export ANTHROPIC_API_KEY=...
python scripts/run_demo.py --llm anthropic
```

Repository map

Path	Purpose
docs/proposal	Problem framing and locked decisions
docs/literature	Reading notes
docs/methodology	Methods and evaluation
docs/thesis	Long-form writing
docs/notes	Working notes and backlogs
contracts	Solidity smart contracts
chain-tests	Hardhat tests for the contracts
research/experiments	Run logs and metrics
research/data	Raw / processed data (local dumps gitignored)
src/core	Hashing and serialisation primitives
src/domain	Log entries, hash chain, Merkle tree, agent, audit verifier (pure)
src/app	Chain client, log storage, demo entrypoint (side effects)
scripts	Thin CLI wrappers, verify_log.py (check a log file), PDF builders
tests	Verification
assets	Figures and diagrams
presentations	Defence decks

Status

Phase: Month 1–2 — working pipeline: demo agent, audit log, anchoring on a local chain, verifier. LLM is a scripted stand-in by default; `--llm ollama` uses a local model, `--llm anthropic` uses Claude.